PROGRAMACIÓN ORIENTADA A OBJETOS II

# Sistema de Gestión

## Soluciones Empresariales S.A.

ANDRES DAVID RESTREPO HERNANDEZ

*Python · MySQL · PyQt6*

| 10 | 5 | 3 |
|---|---|---|
| Módulos BD | Interfaces | Capas |

# CONTENIDO DE LA PRESENTACIÓN


01 **Propósito y Objetivos**
¿Qué queremos lograr?

02 **Problemática**
¿Qué problema resolvemos?

03 **Tecnologías Utilizadas**
Stack técnico del proyecto

04 **Arquitectura del Sistema**
Estructura y capas

05 **Base de Datos**
Esquema y entidades

06 **Interfaces Tkinter**
Módulos desarrollados

07 **Procedimientos Almacenados**
Lógica en MySQL

08 **Conclusiones**
Aprendizajes del equipo

# PROPÓSITO Y OBJETIVOS

## PROPÓSITO

Desarrollar un sistema integral de gestión para la firma Soluciones Empresariales S.A., que centralice la información de clientes, consultores, servicios, proyectos y facturación en una base de datos relacional, accesible desde una interfaz de escritorio construida en Python.

## OBJETIVOS

1. Modelar y crear la base de datos con 10 entidades relacionadas.
2. Implementar la conexión con Python usando POO y mysql-connector.
3. Construir interfaces gráficas con PyQt6 para cada módulo.
4. Aplicar procedimientos almacenados para operaciones críticas.
5. Documentar el sistema y seguir buenas prácticas de desarrollo.

# PROBLEMÁTICA QUE RESUELVE

*Las firmas de consultoría gestionan su información de forma fragmentada — hojas de cálculo, correos y documentos dispersos generan errores e ineficiencias.*

### ⚠️ Sin sistema centralizado

Datos de clientes y proyectos en múltiples archivos sin conexión entre sí.

### ⚠️ Trazabilidad nula

Imposible conocer el estado real de un proyecto o las horas invertidas.

### ⚠️ Facturación manual

Cálculos de honorarios propensos a errores y sin historial confiable.

### ⚠️ Conocimiento no reutilizable

Metodologías y casos de éxito no catalogados ni accesibles al equipo.

# TECNOLOGÍAS UTILIZADAS

### Python 3.11

Lenguaje principal del backend y la interfaz.

### MySQL

Motor de base de datos relacional.

### PyQt6

Biblioteca estándar para GUI de escritorio.

### mysql-connector

Driver oficial de conexión Python ↔ MySQL.

### python-dotenv

Gestión segura de variables de entorno.

### DataGrip

IDE para administración y consulta de BD.

### PyCharm

Entorno de desarrollo para Python.

### Git

Control de versiones del proyecto.

# ARQUITECTURA DEL SISTEMA

## CAPA DE PRESENTACIÓN

**PyQt6**

Ventanas, formularios, tablas y navegación.
Modulos: clientes, consultores, servicios, proyectos, facturas.

↕

## CAPA DE LÓGICA / ACCESO A DATOS

**Python POO**

Clase Database con mysql-connector.
Métodos: conectar(), consultar(), ejecutar().
Repositorios por módulo.

↕

## CAPA DE DATOS

**MySQL**

10 tablas relacionadas + vistas + procedimientos almacenados.
Base de datos: SITEMA_DE_GESTION_SOLUCIONES_EMPRESARIALES.

# BASE DE DATOS — ESQUEMA RELACIONAL

### clientes
código, tipo, razón social, RUC, contacto

### consultores
código, nivel, especialidades, tarifa horaria

### servicios
código, nombre, categoría, metodología

### proyectos
número, cliente FK, servicio FK, estado

### proyecto_consultores
proyecto FK, consultor FK, rol, dedicación

### propuestas
número, cliente FK, inversión, estado

### fases
código, proyecto FK, avance, responsable

### entregables
código, fase FK, tipo, aprobación cliente

### horas_trabajadas
consultor FK, proyecto FK, fecha, horas

### facturas
número, cliente FK, honorarios, total

✦ Incluye 4 vistas preconstruidas: v_resumen_proyectos · v_equipo_proyectos · v_entregables_estado · v_facturacion_clientes

# INTERFACES GRÁFICAS — PyQt6

### 🔗 Conexión

Ventana de login con campos host, puerto, base de datos, usuario y contraseña.

### 🏠 Menú

Acceso central a todos los módulos con botones de colores por sección.

### 👥 Clientes

CRUD completo + búsqueda por nombre, contacto o RUC.

### 👩‍💼 Consultores

CRUD + búsqueda por nombre o especialidad.

### 📋 Servicios

CRUD + búsqueda por nombre, categoría o metodología.

### 📁 Proyectos

CRUD + filtro por estado + vista de equipo asignado por proyecto.

### 🧾 Facturas

CRUD + filtro + cambio rápido de estado de pago.

# CONCLUSIONES Y APRENDIZAJES

### ✦ POO aplicada

Modelar la conexión y los repositorios como clases demostró el valor real de la encapsulación y la reutilización de código frente a scripts procedurales.

### ✦ BD relacional compleja

Diseñar 10 tablas con FK, CHECK constraints, vistas y procedimientos almacenados reforzó el dominio de SQL y la integridad referencial.

### ✦ Separación de capas

Dividir el sistema en Presentación, Lógica/Acceso y Datos facilitó el mantenimiento y permitió cambiar partes sin afectar otras.

### ✦ Flujo de trabajo profesional

El uso de .env, requirements.txt, PyCharm y DataGrip reprodujo un entorno real de desarrollo, acercando el proyecto a la industria.

**¡Gracias!**